

Spam Fighting & Advanced Data Analysis & Visualization with Pandas

Prepared by

Mohammad Almalkawi

PART#1:Spam Fighting System Using Naive Bayes

▪ Dataset overview

For this part, the dataset used was from **the TREC 2007 Spam Corpus**. The collection consists of chronologically ordered emails captured over a period of three months in 2007, labeled with binary annotations.

dataset, which includes about 75,000 email files, categorized as spam or ham, contains 50199 spam emails and 25220 ham emails. - source: kaggle/TREC / G. Cormack

▪ Preparation the code and data

We are import a lot of libraries In order to load and clean the texts, convert them into numerical features, train the model or plot the results.

ToHandles strings, lists, and email-like objects we use the `flatten_to_string` function to simplify nested lists into a single list of strings. email parts into plain text fragments.

Reads an email file and extracts the main text body, cleaning and combining all readable parts into one string by `extract_email_text` function.

After calling the `read_email_files` the function reads all email files from a given directory, extracts their text, and creates two lists: **X** for email texts and **y** for labels (spam/ham).

Function `classify_email` classifies a new email message using the trained model, It converts the raw email text into a numeric feature vector using the trained vectorizer, builds the dataset structure that will be vectorized through.

Loading Dataset and Label Mapping, Load the dataset (emails) and prepare it for preprocessing and model training `DATA_DIR = '../part1/trec07p/data'`

`LABELS_FILE = '../part1/trec07p/full/index'`

, loads all email texts and their corresponding labels from the dataset folder and stores them in `X` and `y`.

Removes HTML tags, numbers, and symbols; converts all text to lowercase to ensure consistency in vectorization

`line = line.strip()` —————> Removes any extra spaces or newline characters at the beginning or end of the line to clean it

`label, key = line.split()` —————> Splits the line into two parts separated by a space

`key.split('/')[-1]` —————> Extracts only the filename from the full path by splitting the string at `/` and keeping the last part ,In order to classify emails as spam or ham

(80:20) train:test ratio

`X_train1, X_test1, y_train1, y_test1 = train_test_split(X, y, train_size=0.8)`

And likewise on (60:40), (50:50), (40:60), (20:80)

We defines multiple experiments train/test ratios to compare model performance under different data size To test and measure the effect of data size on accuracy

Model Training and Evaluation: we Train a Naive Bayes model on each split and evaluate its performance on the vectorized training data.

`vectorizer1 = CountVectorizer()`

`X_train_vector1 = vectorizer1.fit_transform(X_train1)`

The purpose of this section is to convert the training email texts into numerical feature vectors using the CountVectorizer tool.

Each text email is transformed into a numeric representation that can be used by the Naive Bayes model for training

Text Vectorization Convert cleaned email texts into numerical feature vectors using **CountVectorizer()** The CountVectorizer converts each document into a sparse matrix where each column represents a word and each cell shows the word's frequency.

Evaluate the testing accuracy

The model achieves around **95%** accuracy in all splits, showing consistent performance across different training sizes

Average approximate accuracy: 95.36%

This indicates that the model maintained high and stable performance across all different partition ratios.

```
a3='Accuracy {:.1%}'.format(accuracy_score(y_te
a4='Accuracy {:.1%}'.format(accuracy_score(y_te
a5='Accuracy {:.1%}'.format(accuracy_score(y_te
print(a1,a2,a3,a4,a5,sep="\n")
```

```
Accuracy 95.5%
Accuracy 95.6%
Accuracy 95.2%
Accuracy 95.0%
Accuracy 95.5%
```

Testing new messages, Manual test (predict single emails)

The message new_email_spam = "**this is not sus**" It was classified as spam.

The message email_ham = ""**From bounce-debian-mirrors=..... Trouble?**
listmaster@lists.debian.org ""It was classified as ham

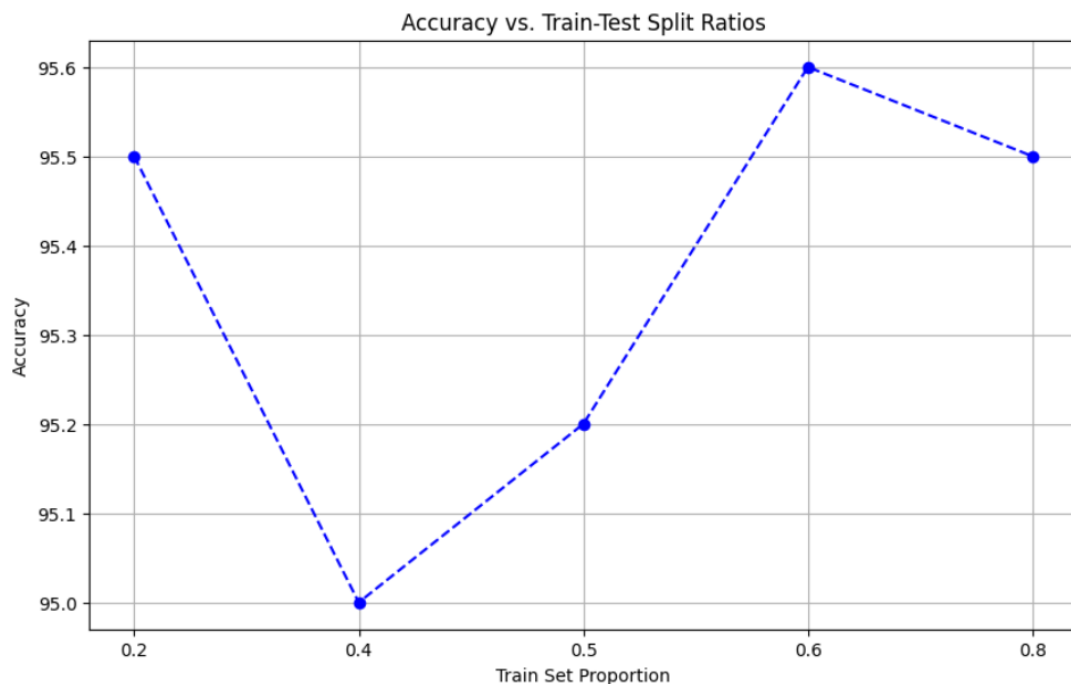
```
print("This email is:", "spam" if classify_email(new_email_spam, mnb1, vectorizer1)==0 else "ham")
print("This email is:", "spam" if classify_email(new_email_ham, mnb1, vectorizer1)==0 else "ham")
```

This email is: spam
This email is: ham

Visualization — Accuracy vs. Train-Test Split Ratios

The purpose of this section is to show visually how changing the proportion of training data affects the accuracy of the model.

It helps in showing if the increased training data improves the performance of the Naive Bayes model.



The graph Line chart shows that the Naive Bayes model maintains a nearly constant accuracy 95% across all the train/test ratios.

That means strong generalization and stability, even with smaller training sets.

Each point represents one experiment result.

These points are connected by the dashed blue line to indicate the trend.

The line is almost flat at 95%, indicating the accuracy of the model performs stably for varying train-to-test ratios.

This suggests that Naive Bayes is consistent even when training data are reduced.

PART#2: Data Analysis & Visualization with Pandas

1. dataset overview :

the dataset we used in this part is AB_NYC_2019.csv it contain information about Airbnb listing in new York city in 2019 , the dataset has 48000 rows and 16 columns .

the dataset has information about new York city and the people who live in the city and other data such as : name, host_id, host_name, neighbourhood, latitude, room_type, price, minimum_nights, number_of_reviews, last_review, reviews_per_month.

The dataset show that there are 221 neighbourhoods, spread over the five main boroughs in the city of new York The neighbourhoods mentioned most in this dataset are: Williamsburg , Bedford-Stuyvesant, Harlem, Bushwick, Upper West Side.

The five areas mentioned above make up a substantial part of all Airbnb listings implying that the two areas Brooklyn and Manhattan are the leading areas that dominate the market regarding the provision of short term accommodation listings offered by Airbnb .

2. Data Cleaning

First we loaded the dataset using pandas library by including this command: `df = pd.read_csv('../part2/AB_NYC_2019.csv')`, then we used `info()` and `describe()` functions to identify missing values, incorrect data types, and duplicates.

Some cleaning procedures were involved:

Missing values: The missing values in numerical columns were replaced by the average or median, respectively, according to the skewness, and those with an excessive number of missing values were removed

Removing Duplicates: The duplicate listings are removed by `drop_duplicates()`.

Type Conversion: Some columns, like dates and numerical identifiers, were changed to their respective types by using `astype()` and `to_datetime()`.

Outlier Identification: The price outliers were identified by box plot analysis, which identified a number of outliers with prices significantly above \$10,000/night

The dataset now ready to be analyzed statistically .

3: Data Exploration and Manipulation

3.1: Price Distribution

first group of operation used to compute the (mean, sum, and median price) for each neighbourhood :

these data from the code

	mean	sum	median	
	neighbourhood			
Allerton	87.595238	3679	66.5	
Arden Heights	67.250000	269	72.5	
Arrochar	115.000000	2415	65.0	
Arverne	171.779221	13227	125.0	
Astoria	117.187778	105469	85.0	
...	...	...	...	...
Windsor Terrace	138.993631	21822	123.0	
Woodhaven	67.170455	5911	52.0	
Woodlawn	60.090909	661	68.0	
Woodrow	700.000000	700	700.0	
Woodside	85.097872	19998	60.0	

It is evident that among all the areas, Astoria and Arverne have some of the highest prices due to their popularity among visitors.

The number of listings is imbalanced, just few neighbourhoods have thousands of listings, and others have less than a dozen It is clear that Airbnb is operational in a particular geographic region or areas around a city.

3.2:Statistical Insights :

we compute the correlation matrix for numerical columns to Identify any strong positive or negative correlations and provide a brief analysis and we created a heatmap of the correlation matrix using Seaborn to identify strong correlations and potential multicollinearity issues.

the skewness of each numeric column calculated to evaluate the normality of the data distribution and it shows :

```
:Skewness for each column
price                2.366408
minimum_nights       2.125566
number_of_reviews    1.258253
reviews_per_month    1.299921
calculated_host_listings_count  2.083859
availability_365     2.350558
```

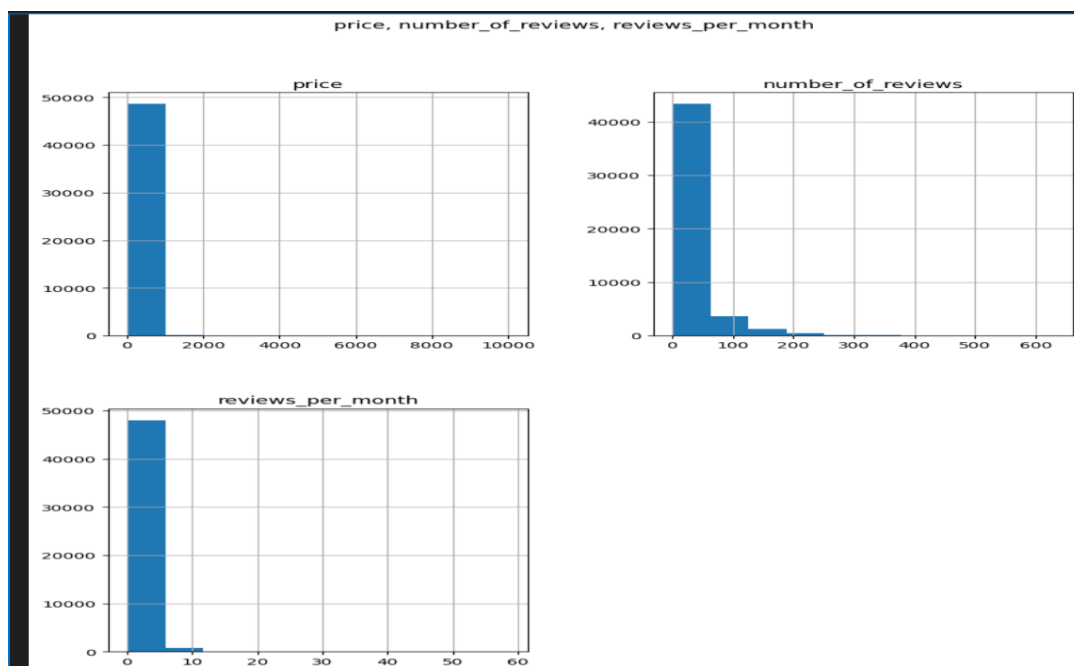
the results show that the value greater than 2 suggests a strongly right skewed distribution indicating that there are mostly small values and a few extremely large values represented by outliers

both price and availability 365 have a heavy tail, meaning that prices are dominated by budget options that are booked for a limited period while a selected few expensive properties are represented.

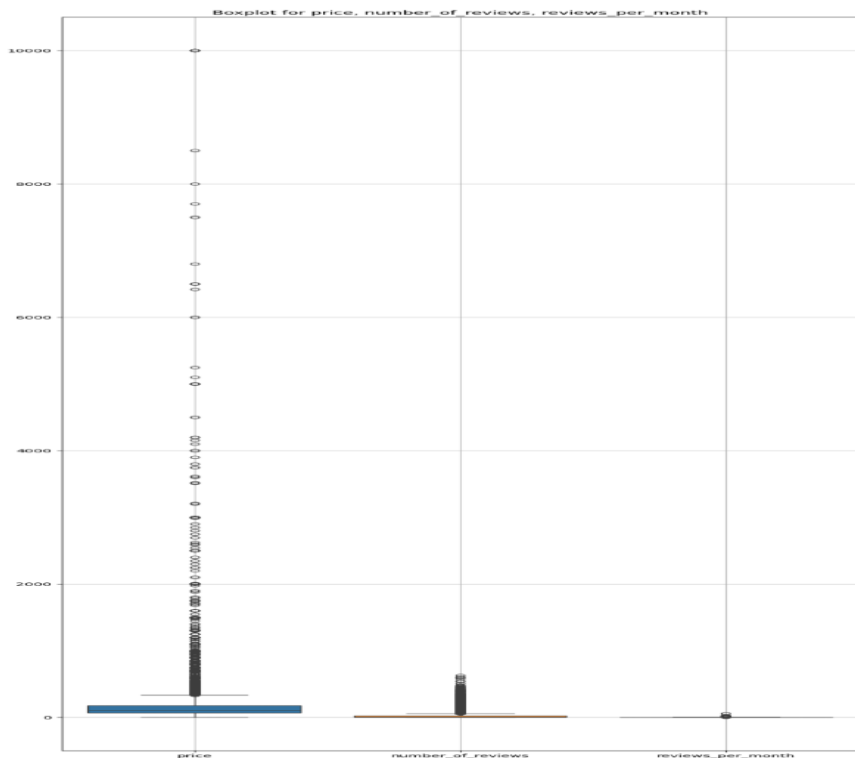
4 . data visualization:

We used several visualizations produced utilizing the visual libraries like: Matplotlib
Pandas Plotting Library

First the histogram graphs were produced for at least three numerical columns such as price, number_of_reviews, reviews_per_month



box plots are used to identify extreme data values and a number of extreme outliers existed in



The scatter plot analysis identify the relation between price and other variables, including ('price', 'number_of_reviews', 'reviews_per_month')



As we see the scatter matrix plot indicated a weak correlation between the numeric variables The price and number of reviews did not show a strong linear correlation implying that a price increase is not necessarily a signal for customer bookings

link for your Jupyter notebooks :

<https://anaconda.com/app/share/notebooks/336944ed-2880-4cd4-b184-e6b1e7f2c81f/overview>

link for the dataset used in part 2 : <https://www.kaggle.com/code/whyalwaysme/ab-nyc-2019>